

The General Intelligence Harness: One Runtime, Instantiated by Manifest

**Memory, Planner, Verifier, Learner, Self-Improvement, Organization,
Self-Model and Mission Manager as the Mechanisms That Earn a Level**

Chengshuai Yang^{1,*}

¹NextGen PlatformAI C Corp, USA

*Correspondence: spiritai@platformai.org

September 3, 2026 — draft v0.1

Abstract

An agent’s intelligence level is a property of a frozen model–harness pair, and the executor is the part of the pair that is easiest to replace. This paper describes the other part: a general intelligence harness whose modules — identity and task state, episodic, semantic and procedural memory, planner, evidence log, verifier, rollback, learner, competence model, self-model, self-improvement, organization and mission manager — are the mechanisms that earn each coordinate of the Unified Intelligence scale $[C, I, O, T, H, SA]$ with its memory prerequisite M . The harness has one runtime instantiated by a manifest that declares which memories exist, what the agent may write, which loops are enabled and which external services evaluate and promote; a domain agent is that runtime plus a domain profile. For each module the paper states in three columns what the agent contains, what must live outside its write set, and the test that promotes it, and maps the module to the harness generation HG0–HG6 that introduces it. Every mechanism that exists is named with its released implementation and its measured status; every mechanism that is only specified is named as specified. The built core — class reading, criterion registration, snapshot and restore, separate-process acceptance, escalation, routing from independent verdicts, compression — has been measured on a harness ladder and on a benchmark whose learning-transfer and self-improvement suites it passed under an ablated control; the mission manager, the organization and the meta-improvement engine are specified and not built. The executor sits behind one protocol, and four executor families are characterised as adapters. The paper’s claim is not that this harness is intelligent; it is that every part of it is either measured or named as unmeasured, and that a reader can build the same thing from the released code.

Keywords: agent harness; memory; planner; verifier; learner; self-improvement; organization; self-model; mission manager; Unified Intelligence

Contents

1	Introduction	2
1.1	What this paper gives	2
2	What a Harness Must Earn	2
3	The Architecture	4
3.1	Memory: three stores and one gate	5
3.2	Planner, verifier interface and rollback: the delegation core	5
3.3	Learner and self-improvement: two different loops	5
3.4	Competence model and router	6
3.5	Self-model	6
3.6	Organization and mission manager	6

4 The Rung Map	7
5 One Runtime, Instantiated by Manifest	7
6 Executors as Adapters	8
7 Built, Measured, Specified	10
8 Apparatus Rules	11
9 Limitations	11
10 Conclusion	11

1 Introduction

When four heavily compared coding agents were placed on the structural axes of an intelligence taxonomy they occupied one cell, because none of them feeds a validated change back into the agent that made it [1]. The difference that would move any of them is not a better model; it is what surrounds the model. This paper is about that surrounding structure, built deliberately.

The design premise comes from the measurement framework of Yang and Xue [7]: a level is earned by measurement of a frozen pair; the architecture enables the claim and the benchmark earns it. The corollary for a harness designer is that every module must be justified by the coordinate it earns and the test that would show it earned it. Modules that earn nothing measurable are decoration; modules whose test cannot be run are hypotheses. This paper writes each module in that discipline.

1.1 What this paper gives

1. **The architecture** (§3): the module set of the general harness, what each contains, what must live outside it, and the test that promotes it.
2. **The rung map** (§4): which generation HG0–HG6 introduces each module, so that a harness scaling curve has a mechanism per step.
3. **The manifest** (§5): one runtime instantiated by declaration, so a domain agent is a profile rather than a program.
4. **Executors as adapters** (§6): one protocol, four executor families, and what each supplies structurally.
5. **Built, measured, specified** (§7): a module-by-module status with the released implementation named where one exists.
6. **The apparatus rules** (§8) that keep the harness’s own measurements honest.

The measurements themselves — scaling curves, matched executor comparisons, transfer — are the companion paper’s [5]; the agents built on this harness and where to download them are the library paper’s [3]. This paper owns the mechanisms.

2 What a Harness Must Earn

The measured profile is $[C, I, O, T, H, SA]$ at reliability p , with M beside I and the one-way gate $I_n \Rightarrow M \geq \mu_n$. The Unified level is a hard gate with retention, and the delegation primitive is defined on delivered outcomes

net of false completions [7]. An LLM can design the mechanism for every level and cannot design three things: the evidence that a level was reached, its own referee, and its own separation [2]. So every module below is stated in three columns — contains, outside, promotes — and “outside” is a deployment fact: a different process with a read-only mount at the weakest, a different host, a different party holding the ground truth, or cryptographic promotion at the strongest. One base model in several processes with separate credentials separates proposer from evaluator; one model in one context, told which hat it wears, separates nothing.

Write-set depth bounds the individual level: D0 nothing persistent, the natural referee; D1 memory; D2 policy; D3 skills; D4 weights. Every module that writes persistently is placed at a depth, and the module that judges it must sit at a lower one.

3 The Architecture

Module	Contains	Must live outside	Promoted by	Earns
Identity and task state	durable identity; goal custody; plan with <i>Verified when</i> : lines; checkpoints	the goal line as authored by the owner	restart continuity test	I1, M1
Episodic memory	what happened, with provenance and temporal order	—	restart plus current-versus-obsolete under distractors	M1, M2
Semantic and procedural memory	consolidated lessons and routines	the held-out family on which a lesson must change behaviour	learning-transfer suite with ablated control; survives an index rebuild	M3, I2
Planner and decomposition	class reading (p, p^*); strategy; one bounded step at a time	the criterion register	delegation surface at declared budgets	T, H
Tools and evidence log	bounded tool grant; append-only log of every action	tool authority as granted	HG0 baseline; false-completion rate	DI baseline
Verifier interface	registration of criteria before the work; sealed, hash-chained; σ reported	the acceptor process and the verdict	paired HG0/HG1 measurement; zero false rejections	acceptance rung
Rollback	snapshot before first mutation; restore on rejection; retry naming the failed check	irreversibility classification	verification responsiveness ρ_V	HG2
Learner	episode $\rightarrow$ candidate lesson $\rightarrow$ consolidation $\rightarrow$ evidence check; NO_CHANGE valid	the evidence check’s criteria	held-out transfer; repeat-error reduction	I2
Competence model and router	$P(\text{verified success} \mid \text{executor, class})$ as a Beta posterior from independent verdicts only; failure classification	the verdicts	routed arm of the arms study; second failure is capability	HG3
Self-model	state, capability limits, authority, freshness; refusals; ledgers never summed	the ground truth of its own state	limits before attempt; mechanism of failure; forecast before run	SA1–SA4
Self-improvement	candidate modification of policy, retrieval, routing, workflow in a sandbox	frozen hidden evaluation; independent evaluator; promoter	hidden improvement with retention; overfit and regressing candidates rejected	I3, HG3 (plan)
Meta-improvement	change ledger; improvement-strategy memory; controlled meta-experiments	the meta-evaluation	Improvement Competence rising across cycles	I4, HG4
Organization	role registry; member roster; evidence-bearing messages; separated proposer, evaluator, promoter	the promoter	ablation: removing one member changes the verified outcome	O1–O3
Mission manager	goal, project and task generators; portfolio state; reprioritization	mission-level evaluator	mission autonomy under changing conditions	T6, HG6
Provenance	hash-chained records; every episode carries why it ended	—	apparatus checks	validity

Table 1: The general harness by module. “Earns” names the coordinate or generation the module’s promotion test certifies. A module with an empty “outside” column is infrastructure the others depend on.

3.1 Memory: three stores and one gate

The memory modules are the temporal substrate of Individual intelligence, and the framework’s rule is that they earn nothing by existing. *M1* is a real process restart followed by recovery of explicitly stored state, with the agent able to say where the state came from. *M2* adds provenance, temporal order and a current-versus-obsolete distinction under distractors. *M3* adds consolidation: a lesson from one episode changes behaviour in a later one and survives a rebuilt retrieval index. Long context is none of these. The gate is one way: an I_n claim is invalid without $M \geq \mu_n$, and an I level may not be promoted because M is high.

Thirty-seven files prove that something wrote thirty-seven files

The fleet’s brain agent carries 37 memory files with provenance markers, and an early edition of its coordinate table entered it as *I1* on that evidence. That reading was withdrawn: the counts are configuration evidence for an *M1* candidate, and the level is settled by stopping the process and starting it again. The learning-transfer suite of the benchmark paper is that test made runnable, and its live run found that the persistence carrying a lesson across a restart was the harness’s own session records, not a store the executor was allowed to write — which is the harness supplying what the executor was denied.

3.2 Planner, verifier interface and rollback: the delegation core

These three modules are the delegation harness, and they are the part of this architecture that is built, released and measured. The planner reads the class before the attempt: how would a mistake be found, what would undoing cost, and therefore what reliability $p^* = \rho / (1 + \rho)$ the class requires; a class whose residual harm is unbounded has $p^* = 1$ and the honest move is to stop and ask. The verifier interface registers criteria before the work exists, refuses a criterion about an existing deliverable, refuses a duplicate, refuses one that does not say what it misses, seals when execution starts, and is hash-chained so an edit is detectable rather than merely discouraged. Acceptance runs in a separate process on a copy of the deliverables with no inherited environment; a check that tries to rewrite the deliverable changes only the copy. Rollback snapshots before the first mutation and restores on rejection, retrying with the failed check named; anything classified irreversible does not run unattended at any capability.

Escalation sits beside them and is preferred to guessing because it has no loss term; the question asked is the shallowest that unblocks. Compression writes out, after a class is accepted, the criteria that accepted it as a reusable check, so the next attempt by any executor arrives with a check that did not exist before; it is the only route by which an agent legitimately raises its own frontier.

3.3 Learner and self-improvement: two different loops

The learner keeps the learning mechanism fixed and lets experience change behaviour: a trigger creates an episode; an episode may generate a candidate lesson; a lesson becomes active only after an evidence check, and `NO_CHANGE` is a valid output. Self-improvement modifies the mechanism itself — a policy, a retrieval strategy, a routing weight — and is well-posed only when the evaluator lies outside the write set: proposer, sandboxed candidate, frozen hidden evaluation, independent evaluator, promoter. The subtlest failure is a candidate that improves the component assembling the evidence the evaluator sees; every standard check passes and the loop self-certifies anyway. The remedy is to freeze the evidence path during evaluation, or to give the evaluator its own.

The promoter’s discrimination is the test of the self-improvement module

A scripted pair that obeys rule lines in its policy and nothing else is given four candidates: the good rules; the dev answers hard-coded; the good rules plus one that breaks a retention item; and no change. A self-improvement module is correct only if it promotes exactly the first and rejects the second and third with the right reasons. The released module does so on 24 of 24 seeds, and in a live run a real pair’s candidate went from 0 to 5 of 5 on hidden instances of a task type absent from its evidence, with retention, and was promoted [6]. One cycle is not *I3*; *I3* needs several, with the mechanism versioned and rollback-able.

3.4 Competence model and router

The router learns $P(\text{verified success} \mid \text{executor, class})$ as a Beta posterior carrying its evidence count beside its mean, updated only from independent verdicts; an executor’s own report of success never enters it. Routing scores $P(\text{success}) \cdot \text{value} - \text{cost} - \text{risk}$ with P taken pessimistically, and an executor whose lower bound cannot reach the class’s p^* is not eligible. Failures re-route by kind: specification back to the contract, execution to the same executor, capability to another (the second failure of one executor is called capability rather than bad luck), environment waits, verification stops. In the arms study the router started with no evidence, watched the verifier reject a hopeless executor twice, and moved the work: 15 of 15.

3.5 Self-model

Operational and evidence-grounded, never phenomenal. Three instruments earn it and each has a floor: limits stated before attempting (the floor is attempting everything and failing); the mechanism of a verified failure named and checked against the runner’s record (“I made an error” fails); a forecast stored by the acceptor with a timestamp before the run, scored by Brier against a constant forecast. The strongest self-model in the fleet keeps three ledgers — owner-set work, benchmark scores, self-directed research — that are never summed, because an agent that wrote its own benchmark, passed it and counted the pass toward its record would have published its own reputation.

3.6 Organization and mission manager

An organization is not a larger individual. *O1* requires a role registry, persistent organizational memory and a dispatched roster; *O2* routing that changes when logged evidence changes; *O3* modification of the organization’s own workflow under external validation. The test that separates an organization from one agent with extra processes is the ablation. From *O2* upward the separation gate applies: proposer, evaluator and promoter are distinct loci and the evaluator’s evidence lies outside the proposer’s write set, or the correct classification is *O1* plus unverified claims. Heterogeneity is required: the natural verifier has an empty or narrow self-write set, and that is its qualification. The mission manager turns a mission into projects, tasks and a portfolio with reprioritization under changing conditions; its input is a mission rather than a task, and its evaluator is mission-level. Neither module is built (§7).

4 The Rung Map

Generation	Modules introduced	Status
HG0	executor, bounded tools, evidence log; no acceptance step	built, measured
HG1	identity and task state; episodic memory; criterion register; separate-process acceptance	built, measured (paired with HG0)
HG2	rollback; retry naming the failed check; learner with evidence-linked consolidation	built; rollback measured, learner measured by the transfer suite
HG3 (as built)	failure classification; competence model; routing	built, measured (arms study; inert on the single-executor ladder)
HG3 (as planned)	self-improvement: sandboxed candidate, frozen hidden evaluation, independent promoter	built as a policy-level suite, one cycle measured
HG4	meta-improvement engine; change ledger; longitudinal project manager	specified
HG5	unknown, hypothesis, experiment and knowledge state; discovery verifier	partly built in the research agents (field map, candidate generator); no sealed certification run
HG6	mission manager; portfolio state; mission-level evaluator	specified
HGΩ	knowledge-to-instrument transduction; new roles under frozen governance	specified

Table 2: Which generation introduces which module, and what exists. The two HG3 rows record a naming split between the ladder as built and the development plan; the companion paper keeps them apart.

Each rung is a strict superset of the one below, enforced by a test, because otherwise a difference between rungs is attributable to nothing. Two rungs that differ only in post-hoc machinery are scored from the same executor outputs.

5 One Runtime, Instantiated by Manifest

A domain agent is $A_d = (m, h, d, B_d, r)$: an executor, this harness, a domain profile, a domain benchmark, and a resource and authority envelope [4]. The harness is one runtime, and the manifest declares which memories exist, what the agent may write, which loops are enabled, and which external services evaluate and promote:

```
agent_id: funding-01      base: general-harness      modes: [CHAT, REASON, ACTION]
state:    episodic_memory: true    semantic_memory: true
          procedural_memory: true   intentional_memory: true
          self_model: true          world_model: true
write_set: memory: true            policy: candidate_only
          skills: candidate_only    objective: false    weights: false
learning: consolidation: true      proceduralization: true
          held_out_ablation_required: true
rsi:      theta_modification: false meta_improvement: false
discovery: unknown_model: false    hypothesis_engine: false
authority: may_execute_tools: allowlisted    may_modify_workspace: false
          may_deploy_self_change: false      outward: none
verifier:  external_required: true           promoter_required: true
```

A different manifest instantiates a D0 verifier, an I2 specialist, an I3 self-improving worker or an I5 discovery agent from the same runtime. Permissions are enforced by infrastructure — file-system permissions, separate accounts, container boundaries, owner-signed promotion — never by instruction, because the model is never shown a permission gate: a restriction honoured by a capable participant is not a mechanism. The placement rule follows: assign each operation the lowest sufficient level, spend the high levels on what only they can do, and put verification in the hands of executors with nothing to write to themselves.

6 Executors as Adapters

The executor sits behind one protocol; the harness sends a contract and receives an episode record and never learns which vendor produced it. A motor is characterised by four things only: what tools it can be granted, whether it can be sandboxed, whether it exposes a headless call, and what it costs.

Executor family	Headless call and what it supplies	Status in this program
Claude Code	<code>claude -p</code> ; ACP bridge; a provenance-bearing memory store; subagents that receive inputs only; permission modes	the reference executor on the ladder and in every suite; carried an open 27B model (Qwen 3.8) through a rewriting proxy to the same surface
Codex	<code>codex exec</code> ; an isolated runner; the exchange-routed fork accepts a base URL	Family B on the ladder
OpenCode	<code>opencode run -m provider/model</code> ; an open harness with its own shell tool; reads its working directory from the environment	ran the T0/T1 families with the free Muse Spark 1.3 model as a third pair
Hermes, Pi, local	adapters written against the protocol, unregistered; each stub announces itself	not installed; an adapter that silently fell back to another vendor would be a lie in the competence model

Table 3: Four executor families as adapters. The harness effect is not a property of one vendor’s model: the same harness carried a commercial and an open model to the same surface [5].

A fifth “executor” is not in the table because it is not a runtime: plain Python under a timer, which is where every model-free step belongs. A step a script can do deterministically must not cost a model call, or the lowest delegation tier is decorative.

7 Built, Measured, Specified

Module	Released implementation	Measured status
Planner (class reading), verifier interface, rollback, escalation, router, compression	ai4science/ harness/ agents/ delegation/ in the AI4Science repository: contract, criterion, reversible, acceptor, escalate, router, compress, run_ladder	three harness scaling curves; paired HG0/HG1 with zero false rejections; arms study 0/15 to 15/15
Identity, task state, memory stores, forecasts, budgets	the sarsi chassis (about seventy modules) in the same tree; the brain agent's store	restart test bound in the benchmark; transfer on two seeds under an ablated control
Learner	consolidate, semantic, playbook in the chassis; improvement.py in the research agents	learning-transfer suite: learned 48/48, ablated 0/48; live transfer on two seeds
Self-improvement	improvement.py, search.py, charter.py (research agents); the benchmark's policy-level suite	promoter sorts good, overfit, regressing and no-change on 24/24 seeds; one live cycle promoted
Competence model and router	executor.py, router.py	routed arm 15/15; inert on a single-executor ladder
Self-model	selfmodel.py, selfaware.py, forecast.py; the research agents' refusing self-model	SA1/SA2 items bound; not yet run on the fleet
Meta-improvement	—	specified
Organization	roster, board, shared in the chassis; no roles/ in any fleet workspace	O0 for every agent; ablation never run
Mission manager	—	specified
Provenance	evidence, ledger, log; the criterion register's hash chain	termination reason now on every episode record

Table 4: What exists and what has been shown. The chassis and research-agent modules are in the public source tree and not yet in the published wheel.

8 Apparatus Rules

Every rule here was violated once in this program and each violation looked like a clean result. An episode’s record carries why it ended, with exit status and whether the limit was reached; a harness-terminated run is `timed_out` or `not_attempted`, never a delivery. Executors run in their own process group and the group is killed, because a detached child holding a pipe turns a timeout into a hang and a hang into a phantom delivery. Every gate is shown to open on a known-passing task before it is trusted to close. A floor is never authored by the agent it scores. Executors that resolve their working directory from the environment must be handed it explicitly. And a probabilistic trap must be shown to fire on every instance, or the suite fills silently with items that discriminate nothing.

9 Limitations

- **Three modules are specified and not built:** meta-improvement, mission manager, and the organizational structure beyond a roster file. Every level that depends on them — *I4*, *O1* and above, *T6* — is unreachable by this harness today.
- **The self-improvement module has been exercised on the shallowest writable object**, a policy file. Retrieval and routing candidates need the evaluator’s evidence path frozen, which has not been done.
- **Every measurement cited is from one host and one operator**, on suites that are saturated for frontier executors above the acceptance rung.
- **The published wheel is stale.** The chassis and the research-agent modules install only from a source checkout.

10 Conclusion

The harness is the part of the pair that earns the level, and this paper has written it module by module in the only form the framework accepts: what the module contains, what must live outside it, and the test that promotes it. The delegation core is built and measured. Memory, learning and policy-level self-improvement are built and have passed their suites under ablated controls. Meta-improvement, organization and the mission manager are specified, and the paper says so rather than describing them in the present tense. One runtime, instantiated by manifest, is what makes a domain agent a profile rather than a program — and the library paper is where the profiles are.

Availability

The delegation harness, the chassis and the research-agent modules are in <https://github.com/integritynoble/AI4Science>. The benchmark suites that measure them, with run records, are in <https://github.com/integritynoble/sarsi-intelligence-level> under `uab_v01/`.

Conflicts and funding

The author reports no funding specific to this work and no competing interest.

References

- [1] Chengshuai Yang. Capability is not a level: A structural taxonomy of individual and organizational intelligence, with placements for current AI agents. Draft, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.
- [2] Chengshuai Yang. Designing LLM-driven agents across individual and organizational intelligence levels. Working paper and build specification, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.

- [3] Chengshuai Yang. The domain agent library: Six agents on one harness. Draft v0.1, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.
- [4] Chengshuai Yang. Unified agent intelligence development plan: Benchmark-first development of general agents before AI4Science integration. Planning document, September 2026, 2026.
- [5] Chengshuai Yang. Engineering higher-intelligence AI agents: Harness scaling across models and domains. Draft v0.1, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.
- [6] Chengshuai Yang. Unified agent benchmark v0.1: Specification, coverage, and what is bound. Draft v0.1, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.
- [7] Chengshuai Yang and Ting Xue. Measuring the unified intelligence level: A cumulative, harness-measurable hierarchy. *Version 2.3*, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.